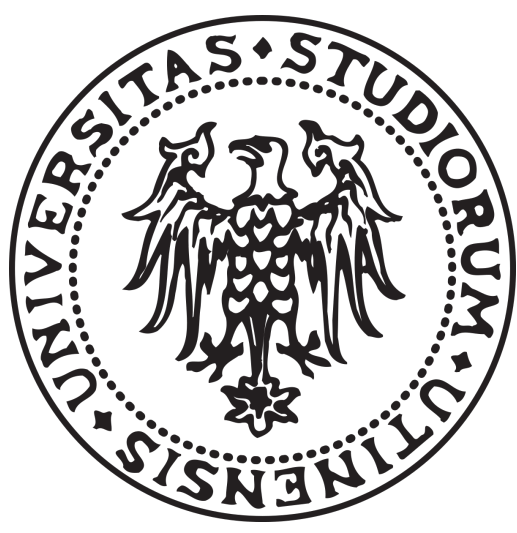




# INGEGNERIA DEL SOFTWARE - 2025-26

---

## DIAGRAMMA DELLE CLASSI: ASPETTI AVANZATI

LEZIONE 15  
17/11/2025  
VINCENZO RICCIO

# RIFERIMENTI

- Fowler - Capitoli 3, 5

Vincenzo Riccio  
Ingegneria del Software 2025/2026  
Università degli Studi di Udine





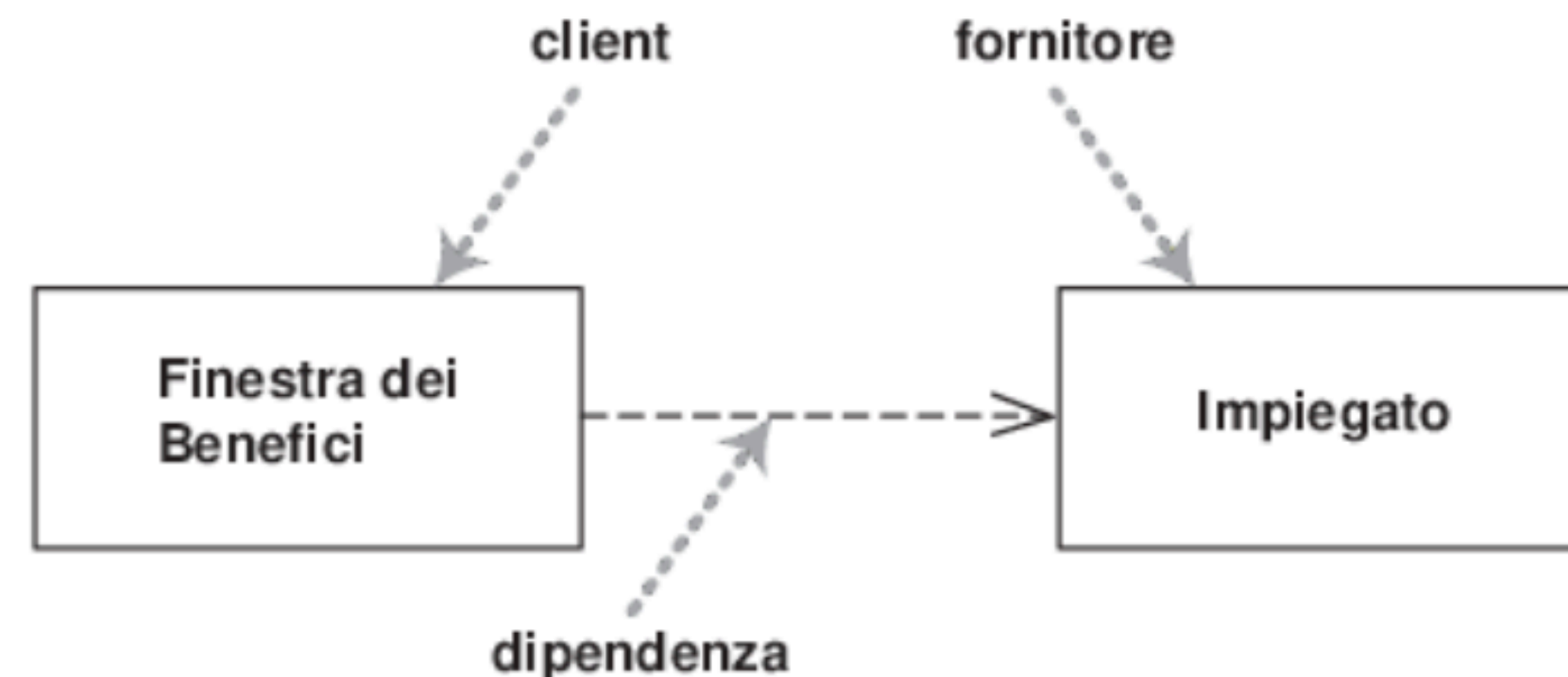
# DIPENDENZE

# DIPENDENZA (DEPENDENCY)



- La Dependency si rappresenta con una linea tratteggiata **unidirezionale** che termina con una freccia dall'oggetto dipendente (client) verso quello da cui dipende (detto provider o fornitore)
- Specifica i punti in cui modificare una classe fornitore potrebbe coinvolgerne altre (client)

MODIFICHE ALLA  
FINESTRA NON HANNO  
EFFETTO SULLA  
CLASSE IMPIEGATO



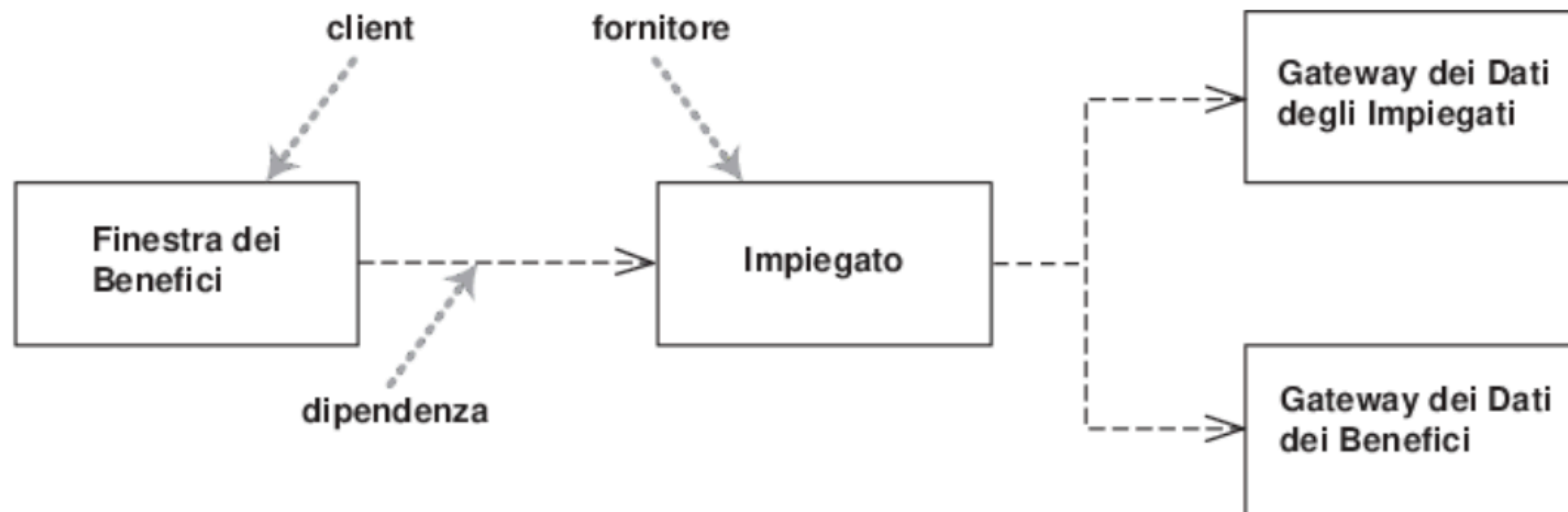
SE CAMBIA LA CLASSE  
IMPIEGATO, DEVE CAMBIARE  
ANCHE LA FINESTRA  
DELL'INTERFACCIA GRAFICA

Esempio di separazione tra  
Presentazione e Business Logic

# DIPENDENZA: PROPAGAZIONE DEL CAMBIAMENTO



- Finestra dei benefici non ha dipendenza diretta dalle Gateway dei dati
- **Non transitività delle dipendenze:** modifiche nelle Gateway comportano modifiche in Impiegato ma non dovrebbero comportare modifiche nella Finestra dei Benefici se la modifica interessa solo l'implementazione di Impiegato e non la sua interfaccia



# DIPENDENZA (DEPENDENCY)



- Si ha una dipendenza tra due classi se la modifica alla definizione della classe provider può causare un cambiamento alla classe client
- Esempi:
  - ⦿ Un'operazione del client invoca un'operazione del provider
  - ⦿ Il provider è il tipo di un'attributo del client
  - ⦿ Il provider é un parametro o il tipo di ritorno di un'operazione del client
- **Problema:** se il provider viene modificato e cambia la sua interfaccia, tale modifica deve coinvolgere anche il client -> *effetto domino* per sistemi complessi per cui le modifiche potrebbero diventare onerose

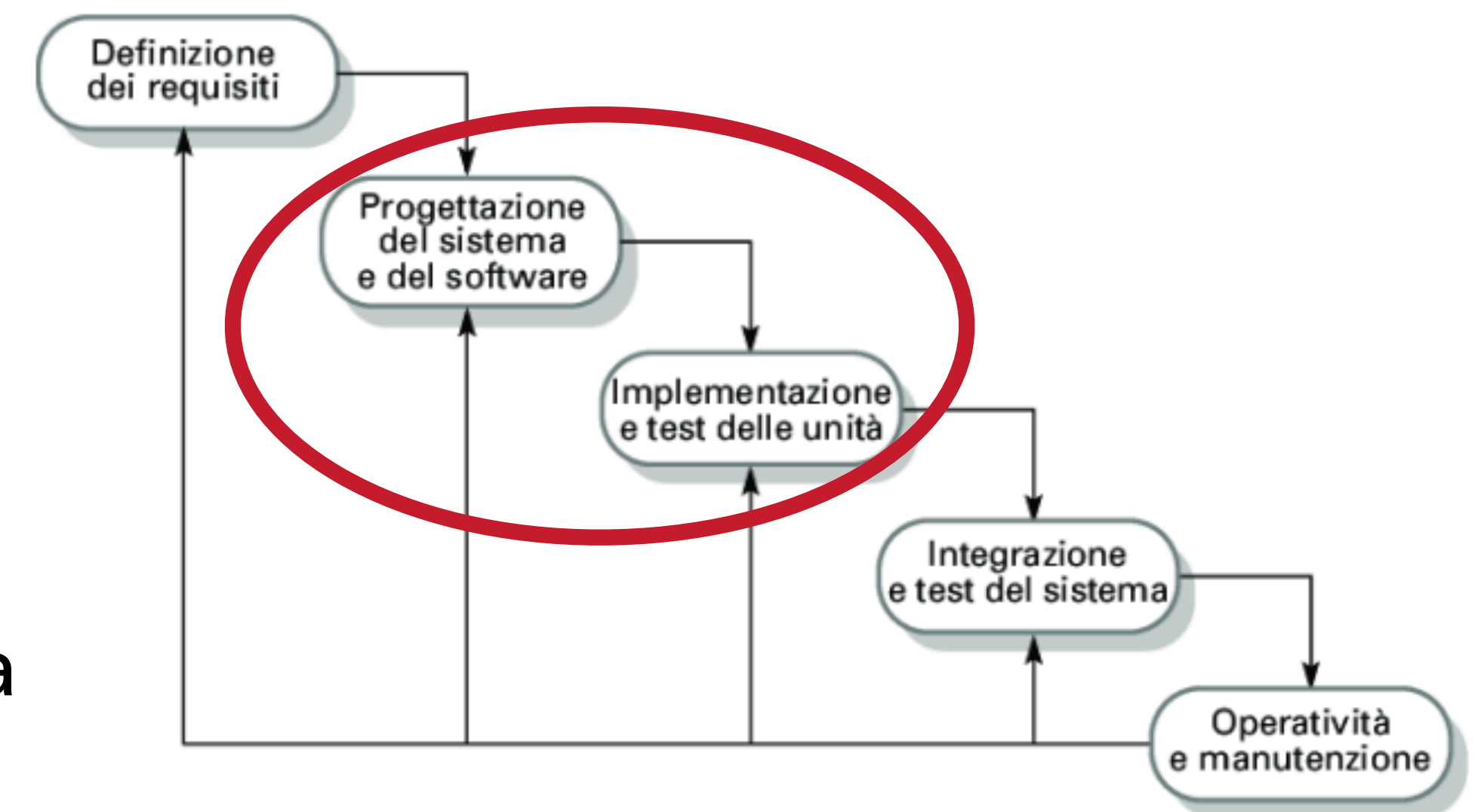


- UML prevede diverse tipologie di dipendenza
- Per distinguere la tipologia di dipendenza, UML mette a disposizione gli stereotipi, indicati con keywords tra parentesi angolari
- Esempi:
  - ◎ **<<use>>**: La sorgente (client) effettua un generico uso di qualcosa della destinazione (provider) **[dipendenza d'uso generica]**
  - ◎ **<<call>>**: La sorgente (client) invoca un'operazione del provider
  - ◎ **<<create>>**: La sorgente crea istanze della classe del provider

# IMPLICAZIONI DELLE DIPENDENZE



- Le dipendenze sono individuate solitamente durante le fasi di progettazione di dettaglio, raramente contribuiscono al modello concettuale
  - Durante il testing, la correttezza della classe client è vincolata dalla correttezza della classe provider
  - La classe client non può essere riusata senza la classe provider
  - Modifiche nella classe provider, implicano modifiche nella classe client (problema ripple effect: se ci sono troppe dipendenze, modificare una classe implica modifiche in molte altre classi)



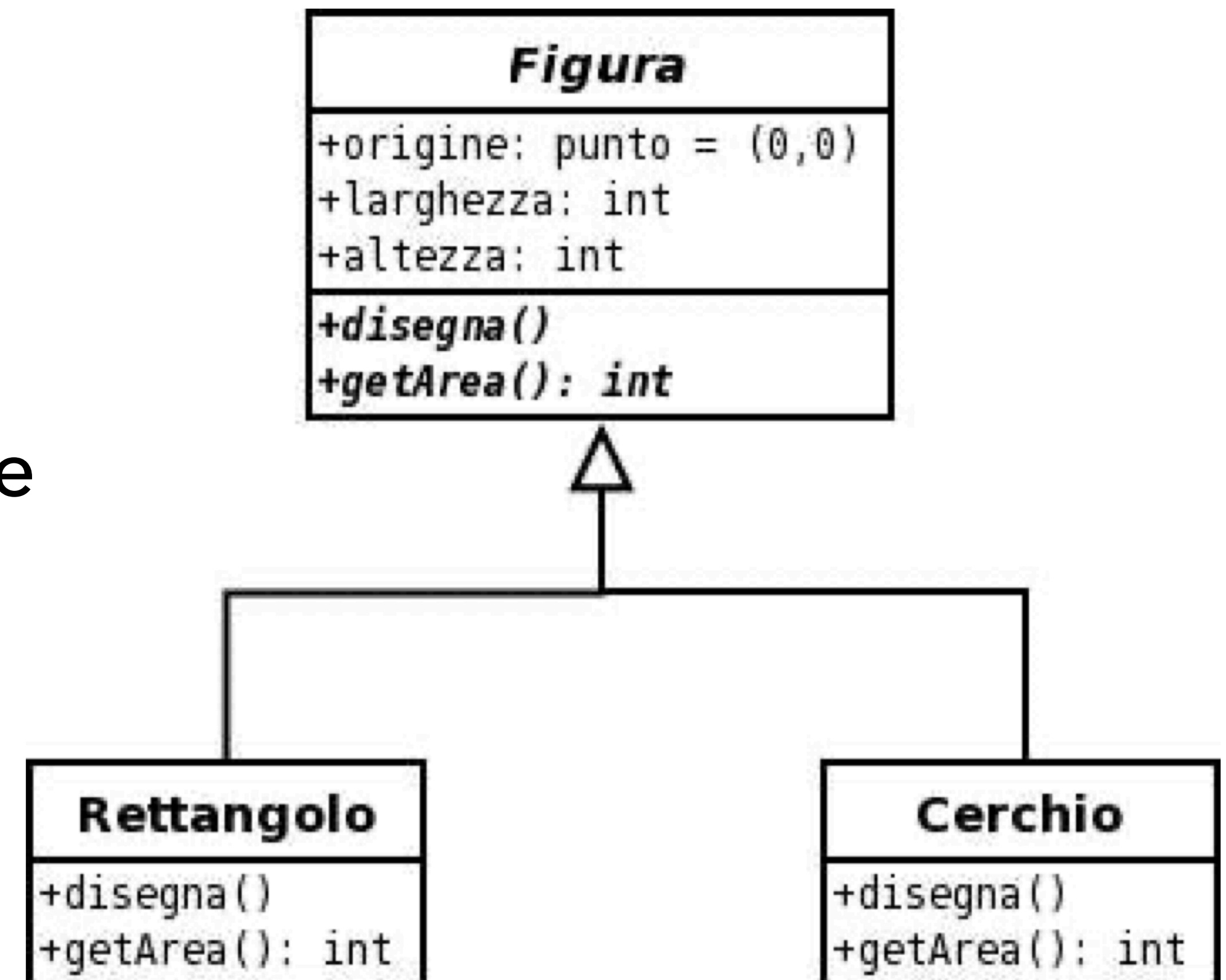


# INTERFACCE E CLASSI ASTRATE

# CLASSI ED OPERAZIONI ASTRATTE

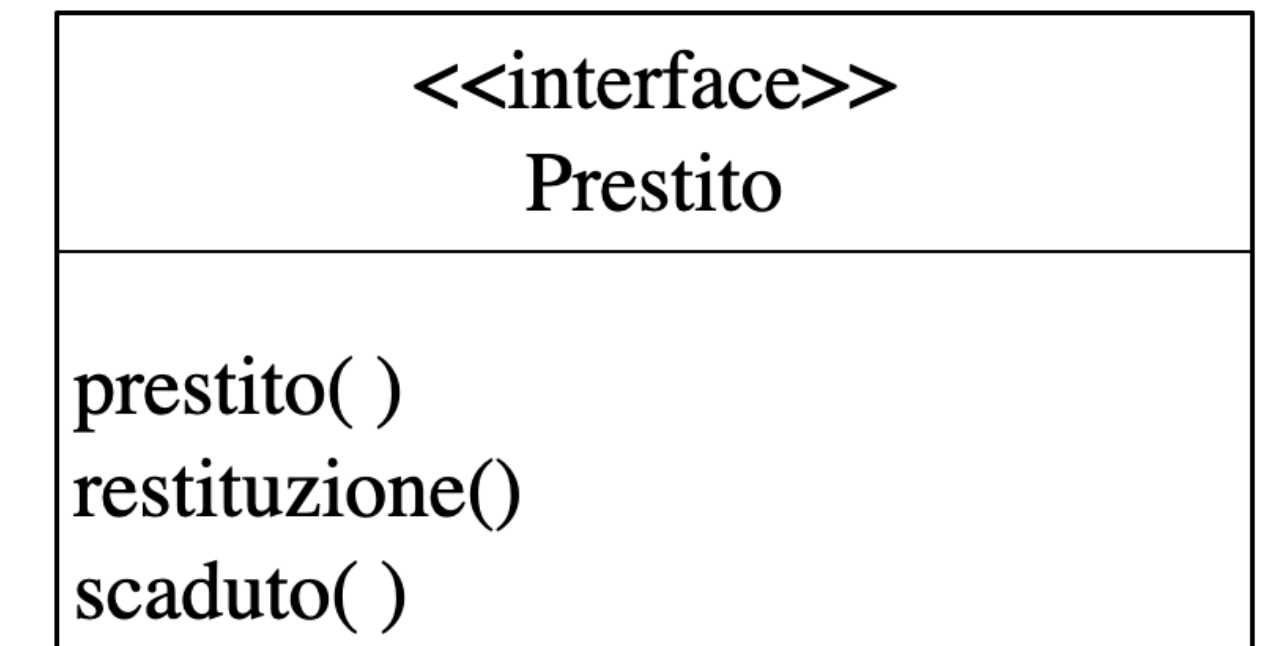


- Una classe astratta è una classe che non può essere direttamente istanziata poiché ha **una o più operazioni astratte** (solo dichiarazione pubblica, senza implementazione)
  - L'implementazione è demandata alle sottoclassi
- Per rappresentare una classe o operazione astratta in UML si scrive il nome in corsivo o si usa l'etichetta {abstract} (molto più leggibile)

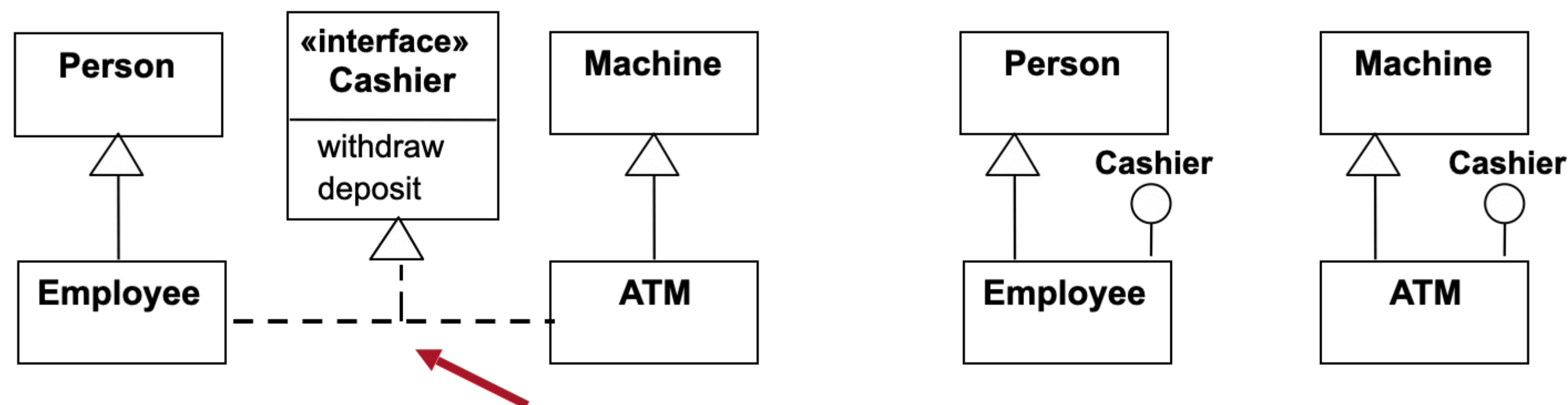




- Classe priva di implementazione (**tutte le operazioni sono astratte**)
- Descrive una porzione di comportamento visibile comune a un insieme di oggetti
- Una o più classi possono fornire l'implementazione dell'interfaccia
- In UML si usa una notazione grafica simile alla classe ma con la parola chiave **<<interface>>**
- Le classi che implementano un'interfaccia sono obbligate a fornire un'implementazione per tutti i metodi dichiarati nell'interfaccia



- L'idea di base è di separare la specifica di una funzionalità (l'interfaccia) dall'implementazione della stessa da parte di un classe (o un sottosistema).
- Una o più classi possono fornire l'implementazione (o realizzazione) dell'interfaccia.
- Due possibili rappresentazioni:



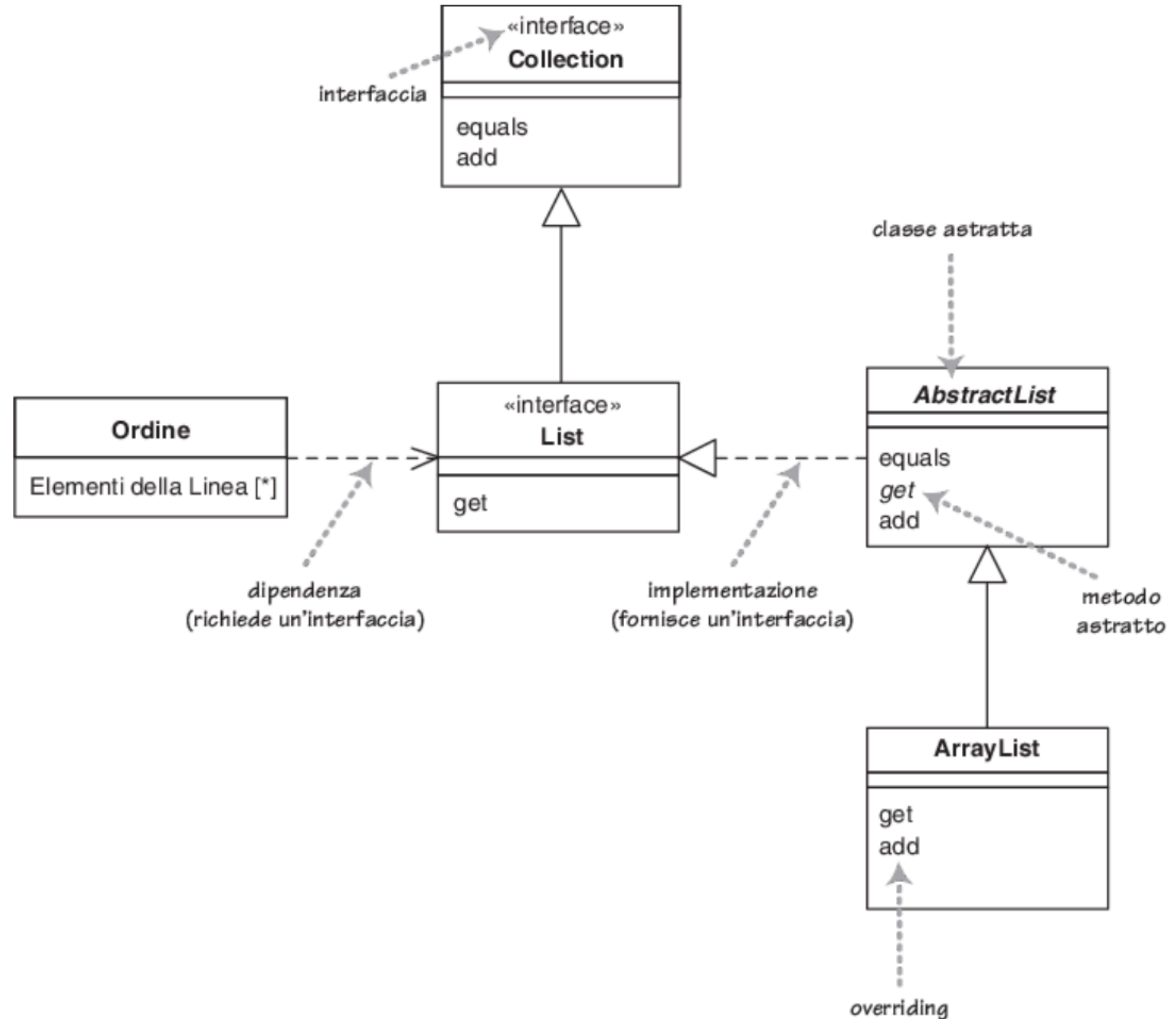
relazione di realizzazione tra una  
classe concreta ed una interfaccia



- Le classi hanno due tipi di rapporti con le interfacce:
  - ⦿ Una classe **può fornire** un'interfaccia se è sostituibile ad essa (in pratica la implementa)
  - ⦿ Una classe **può richiedere** un'interfaccia se ha bisogno di una sua istanza per funzionare (è una forma di dipendenza)

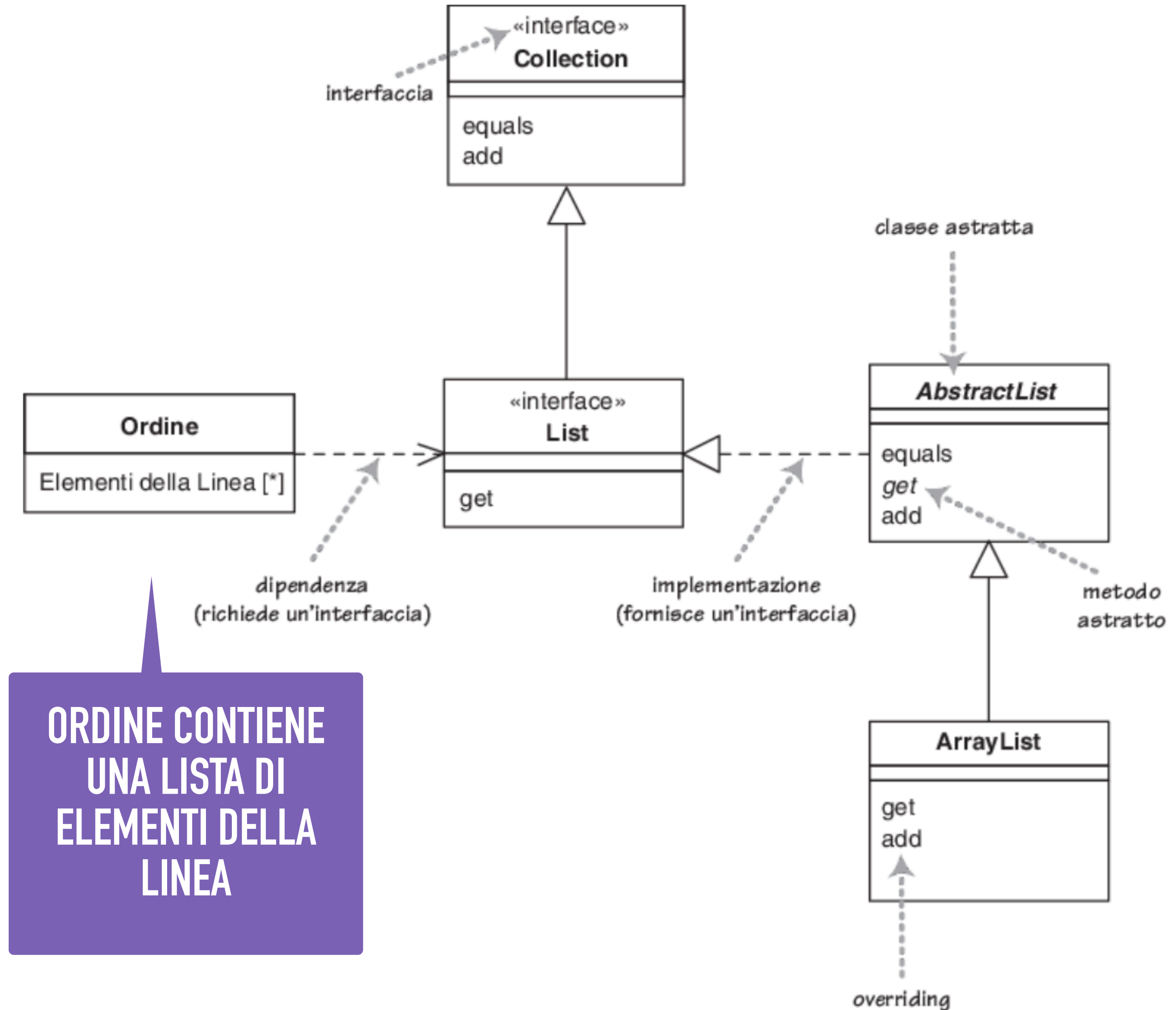
# CLASSI E INTERFACCE

- Esempio con due interfacce e una classe astratta



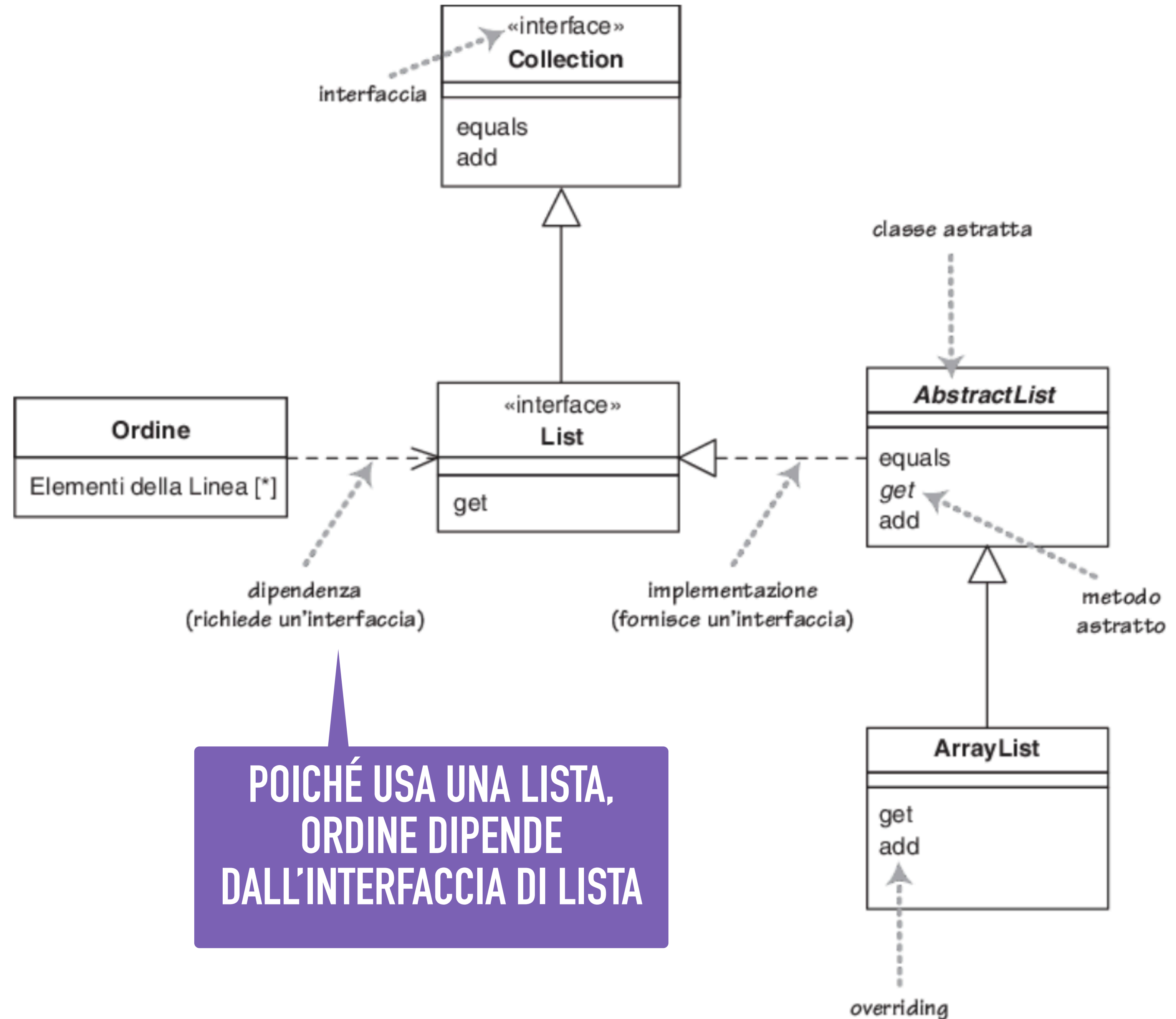
# CLASSI E INTERFACCE

- Esempio con due interfacce e una classe astratta



# CLASSI E INTERFACCE

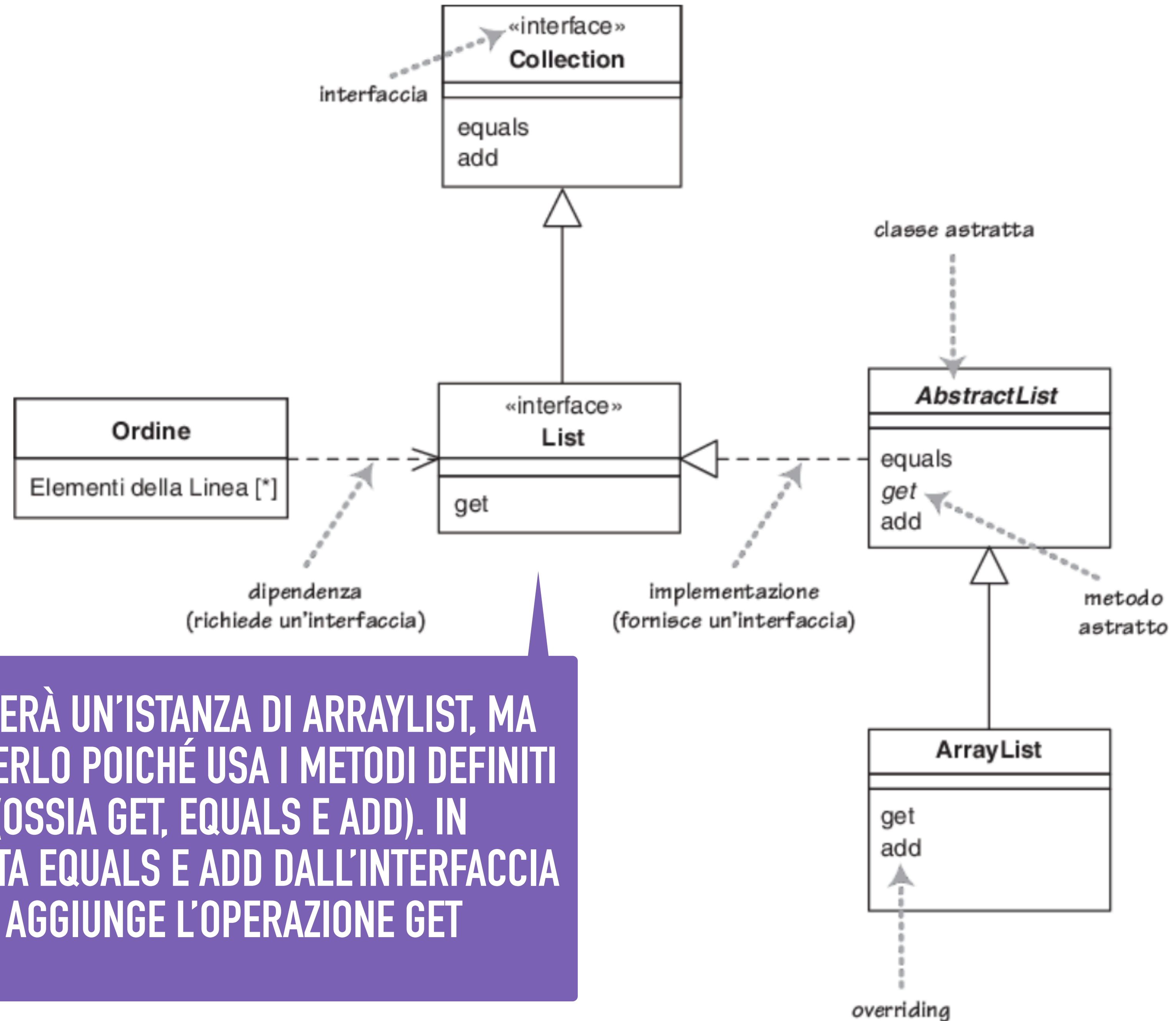
- Esempio con due interfacce e una classe astratta



POICHÉ USA UNA LISTA,  
ORDINE DIPENDE  
DALL'INTERFACCIA DI LISTA

# CLASSI E INTERFACCE

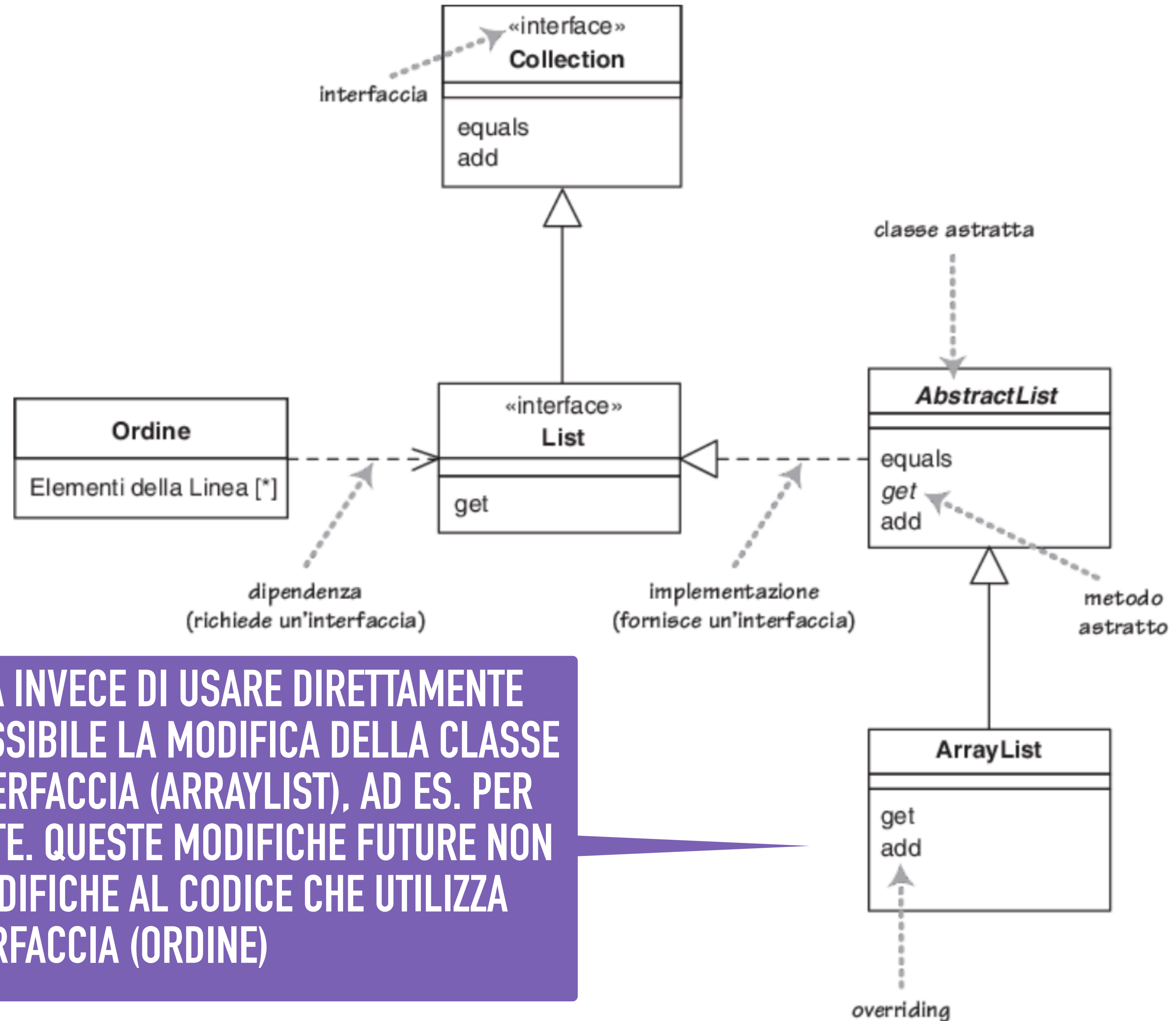
- Esempio con due interfacce e una classe astratta



L'ISTANZA DI ORDINE USERÀ UN'ISTANZA DI ARRAYLIST, MA NON HA BISOGNO DI SAPERLO POICHÉ USA I METODI DEFINITI NELL'INTERFACCIA (OSSIA GET, EQUALS E ADD). IN PARTICOLARE LIST EREDITA EQUALS E ADD DALL'INTERFACCIA COLLECTION, A CUI AGGIUNGE L'OPERAZIONE GET

# CLASSI E INTERFACCE

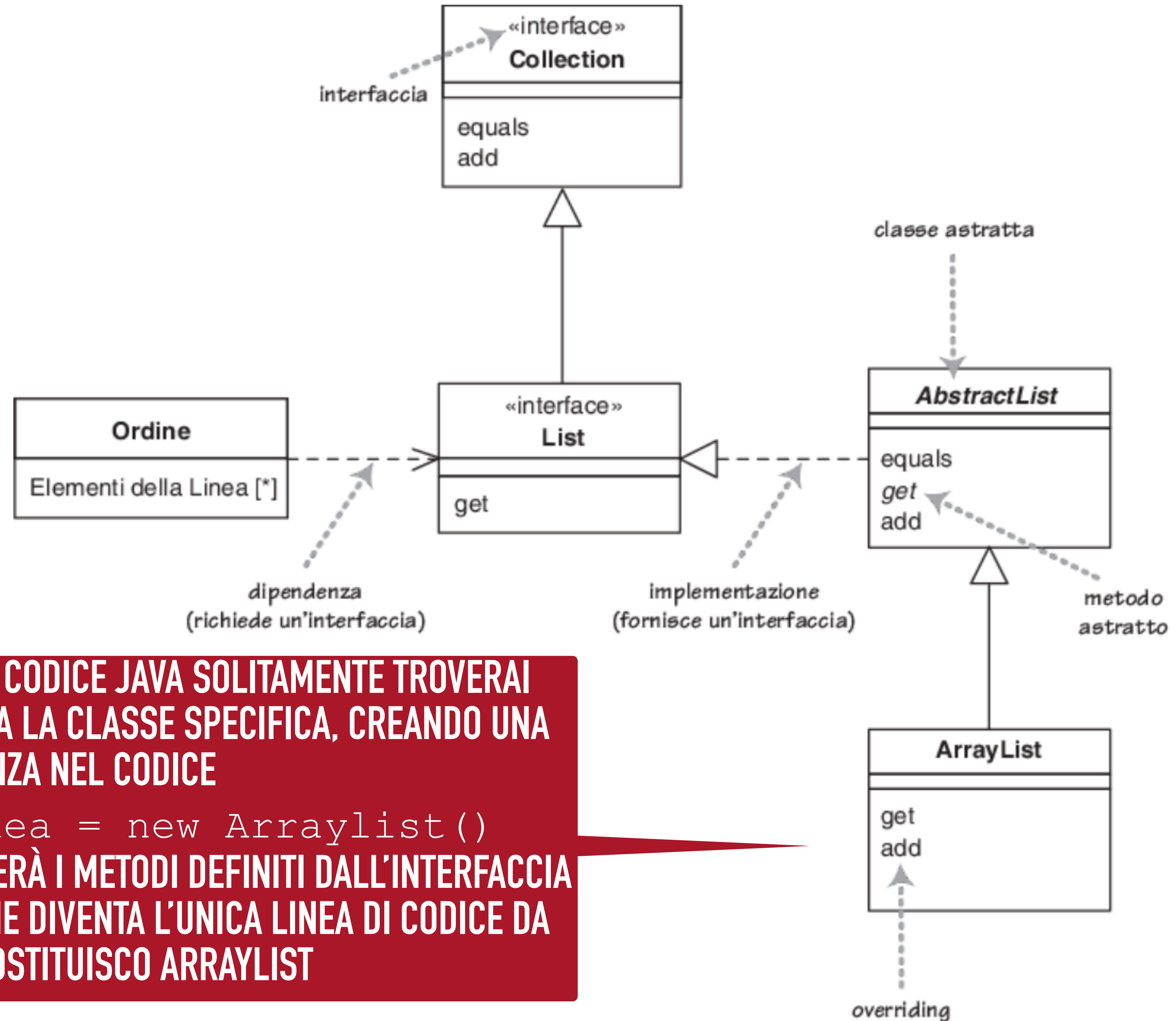
- Esempio con due interfacce e una classe astratta



USANDO L'INTERFACCIA INVECE DI USARE DIRETTAMENTE ARRAYLIST, SI RENDE POSSIBILE LA MODIFICA DELLA CLASSE CHE IMPLEMENTA L'INTERFACCIA (ARRAYLIST), AD ES. PER RENDERLA PIÙ EFFICIENTE. QUESTE MODIFICHE FUTURE NON COMPORTERANNO MODIFICHE AL CODICE CHE UTILIZZA L'INTERFACCIA (ORDINE)

# CLASSI E INTERFACCE

- Esempio con due interfacce e una classe astratta



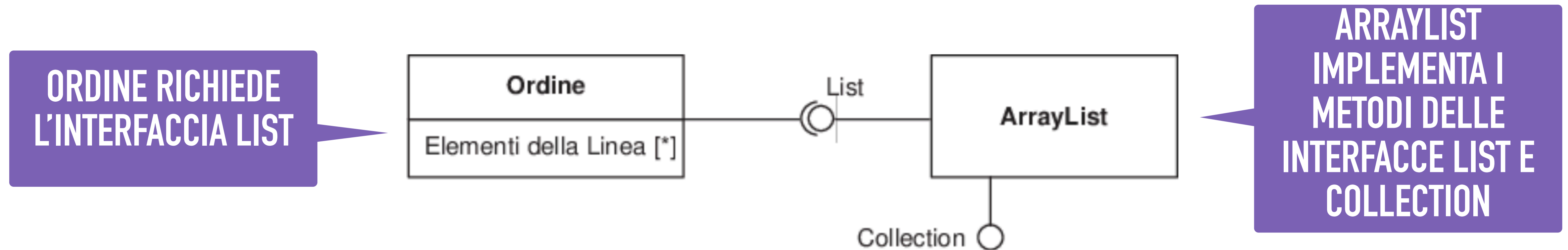
**PICCOLA CONTRADDIZIONE: NEL CODICE JAVA SOLITAMENTE TROVERAI L'INIZIALIZZAZIONE CHE MENZIONA LA CLASSE SPECIFICA, CREANDO UNA DIPENDENZA NEL CODICE**

`List Elementi_linea = new ArrayList()`  
**TUTTAVIA IL RESTO DEL CODICE USERÀ I METODI DEFINITI DALL'INTERFACCIA E QUINDI QUESTA INIZIALIZZAZIONE DIVENTA L'UNICA LINEA DI CODICE DA CAMBIARE SE SOSTITUISCO ARRAYLIST**

# NOTAZIONE COMPATTA LOLLIPOP



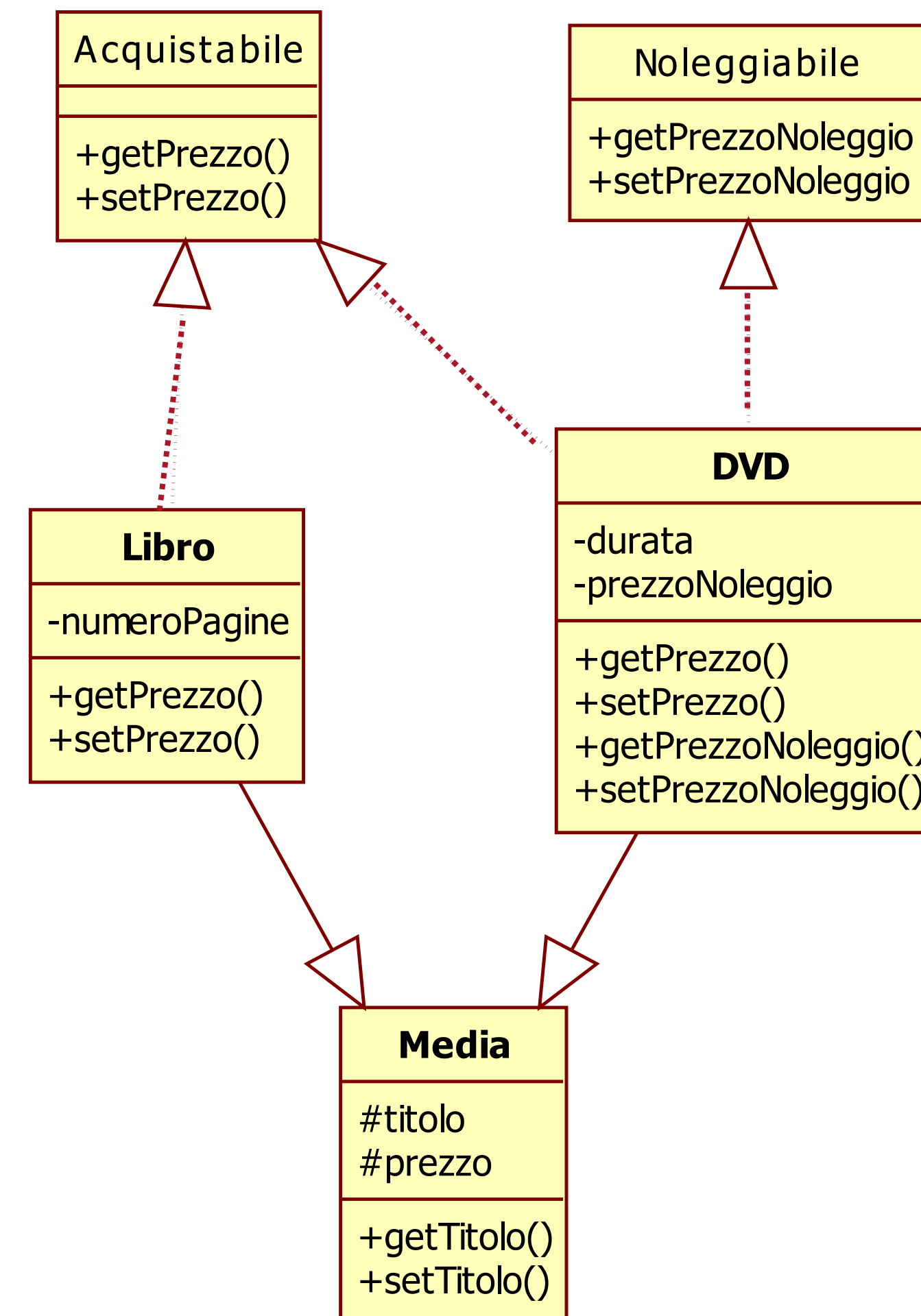
- Una notazione molto utilizzata per le interfacce è quella "Lollipop":
  - ⊙ La **pallina (lollipop)** rappresenta l'interfaccia esposta da una classe (quindi è una realization)
  - ⊙ Il **semicerchio (socket)** rappresenta il servizio richiesto (quindi è una dipendenza)
- L'esempio precedente può essere rappresentato in forma più compatta:



# ESEMPIO



- Un libro è (implementa l'interfaccia) acquistabile (dobbiamo sapere/modificare il suo prezzo)
- Un DVD è acquistabile ma anche Noleggiabile (dobbiamo sapere/modificare il suo prezzo di noleggio)
- Libri e DVD sono entrambi media, con un titolo e un prezzo





```
public interface Acquistabile {
    public void setPrezzo(double
    prezzo);
    public double getPrezzo();
}

public interface Noleggiabile {
    public void setPrezzoNoleggio(double
    prezzo);
    public double getPrezzoNoleggio();
}

public class Media {
    protected String titolo;
    protected double prezzo;
    public void setTitolo(String titolo) {
        this.titolo = titolo;
    }
    public double getTitolo() {
        return titolo;
    }
}
```

```
public class Libro extends Media implements Acquistabile
{
    private int numeroPagine;
    public void setPrezzo(double prezzo) {
        this.prezzo = prezzo;
    }
    public double getPrezzo() {
        return prezzo;
    }
}

public class DVD extends Media implements Acquistabile,
Noleggiabile {
    private double durata;
    private double prezzoNoleggio;
    public void setPrezzo(double prezzo) {
        this.prezzo = prezzo;
    }
    public double getPrezzo() {
        return prezzo;
    }
    public void setPrezzoNoleggio(double prezzo) {
        this.prezzoNoleggio = prezzo;
    }
    public double getPrezzoNoleggio() {
        return prezzoNoleggio;
    }
}
```

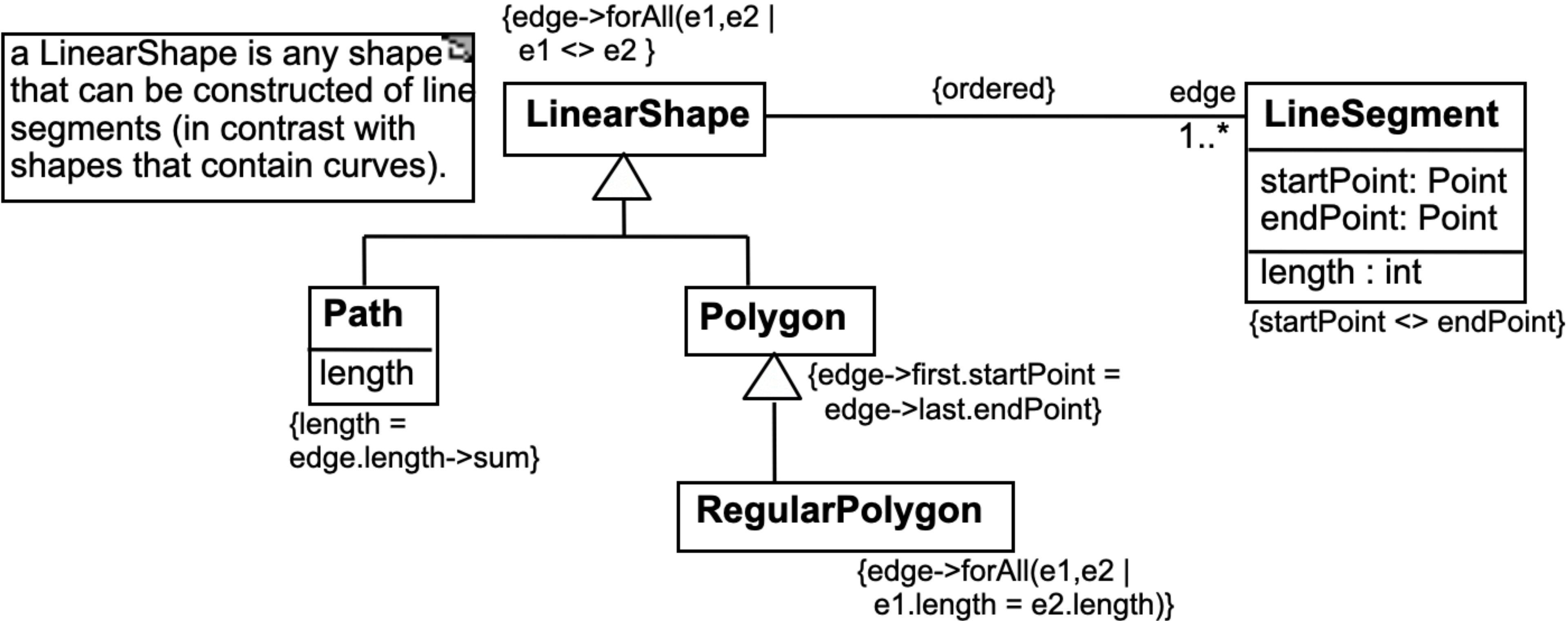


# VINCOLI



- Attraverso associazioni, molteplicità, attributi, generalizzazione si esprimono i vincoli più importanti delle classi (ad es. un automobile può essere proprietà di una sola persona)
- Alcuni vincoli non possono essere espressi dai costrutti base del diagramma
- UML permette la definizione di ulteriori vincoli nel diagramma delle classi **tra parentesi graffe**, in diversi linguaggi:
  - Linguaggio naturale
  - Linguaggio di programmazione
  - Linguaggi formali specifici, come OCL

# ESEMPIO DI VINCOLI



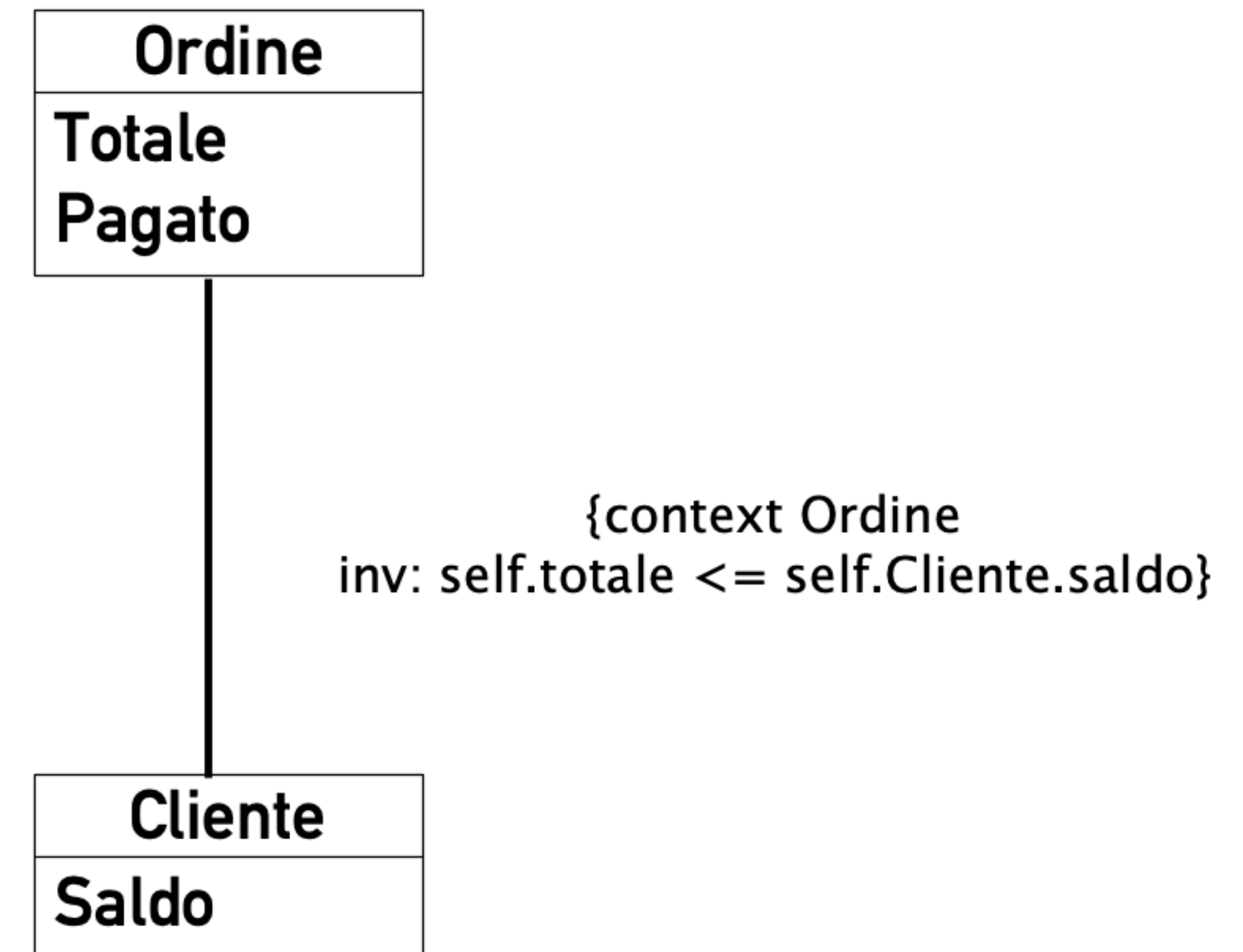


- **Object Constraint Language (OCL)** é un linguaggio di specifica progettato per specificare vincoli nei moduli software
- Una espressione OCL specifica un vincolo (predicato logico) sul sistema che deve assumere valore vero
- Gli statements OCL nei diagrammi delle classi possono specificare quali devono essere i valori di attributi e associazioni: il programmatore dovrà poi rispettare tali vincoli

# OBJECT CONSTRAINT LANGUAGE



- I predicati OCL sono costruiti in tre parti:
  - ◎ **Contesto:** Specifica l'elemento del modello UML a cui si applica (es. classe, associazione, attributo).
  - ◎ **Regola:** Specifica il tipo di vincolo. Può essere invariante **inv** (deve essere sempre vero), preconditione **pre** (deve essere vero prima dell'esecuzione di un'operazione della classe), postcondizione **post**
  - ◎ **Espressione:** Definisce il vincolo
- Sintassi:  
context <Contesto>  
<Regola>: <Espressione>





- Un invariante è un vincolo che deve essere vero per tutta la vita del suo contesto
- Es.: La fine di un meeting deve sempre essere successiva all'orario di inizio
  - ⊙ `context Meeting inv: self.end > self.start`
- Una preconditione deve essere vera prima dell'esecuzione di un'operazione, determinando l'applicabilità di un'operazione
- Es.: Un meeting può essere spostato di N giorni se esso non è stato confermato e se il numero di giorni è un numero maggiore di 0
  - ⊙ `context Meeting::shift(day:Integer) pre: self.isConfirmed = false and day>0`
- Una postcondizione deve essere vera dopo l'esecuzione di un'operazione. Specifica cosa fa un'operazione, non come la fa
- Es.: Dopo la conferma del meeting, il suo attributo `isConfirmed` dovrà essere `true`
  - ⊙ `context Meeting::confirm() post: self.isConfirmed = true`



- **Vantaggi:**
  - ⦿ La notazione formale evita le ambiguità del linguaggio naturale
  - ⦿ Dai vincoli formali è possibile generare automaticamente casi di test
- **Svantaggio**
  - ⦿ OCL può generare incongruenze se chi lo scrive o chi legge non conosce bene OCL
- **Nota:** L'utilizzo di self, come negli esempi precedenti è opzionale ma permesso



# SOLUZIONE ESERCIZIO



- Si consideri una classe ContoBancario con un'operazione `preleva(importo: Double)` che consente di prelevare una somma di denaro dal conto. Si vuole rappresentare con una preconditione OCL che l'importo richiesto dall'operazione `preleva` non superi il saldo disponibile sul conto (`saldo`) e che sia maggiore di zero.
- ```
context ContoBancario::preleva(importo: Double)
pre: importo > 0 and importo <= self.saldo
```